



DICE: Enabling Efficient General-Purpose SIMT Execution with Statically Scheduled Coarse-Grained Reconfigurable Arrays

Jiayi Wang
University of Washington
Seattle, WA, USA
jwang710@uw.edu

Ang Da Lu
University of Washington
Seattle, WA, USA
angl7@uw.edu

Zhichen Zeng
University of Washington
Seattle, WA, USA
zczeng@uw.edu

Ang Li
University of Washington
Seattle, WA, USA
angliz@uw.edu

Abstract—While general-purpose graphics processing units (GPGPUs) dominate massively parallel computing through the single-instruction, multiple-thread (SIMT) programming model, their underlying single-instruction, multiple-data (SIMD) execution incurs substantial energy overhead from frequent register file (RF) accesses and complex control logic.

We present DICE, a novel architecture that addresses these inefficiencies by replacing the SIMD backend with minimal-overhead, statically scheduled coarse-grained reconfigurable arrays (CGRAs) while preserving the well-known SIMT programming model. Unlike SIMD units that execute warps of threads in lockstep, DICE dispatches active threads in a pipelined manner onto the CGRA fabric, where data flows directly between processing elements (PEs), reducing RF accesses for intermediate values. Moreover, DICE organizes threads into groups larger than warps, reducing overhead by eliminating the warp-level control logic and further amortizing remaining control overhead across hundreds of threads.

To handle operations with runtime dynamism, such as variable-latency memory loads and data-dependent control flow, while preserving static scheduling, DICE compiles programs into *p-graphs* by partitioning dynamic dependence edges across separate CGRA configurations. DICE further introduces several key optimizations: double-buffered configuration memory to hide reconfiguration latency, compile-time *p-graph* unrolling to enhance resource utilization, and a temporal memory coalescing unit (TMCU) to merge memory requests from consecutive, pipelined threads. Evaluations on Rodinia benchmarks in Accelsim demonstrate that DICE reduces register file accesses by 68% on average. With equivalent computation and memory resources, DICE's CGRA Processors (CPs) achieve a geometric mean of $1.77\text{--}1.90\times$ dynamic energy efficiency and 42.0%–45.9% average power reduction compared to the modeled NVIDIA Turing Streaming Multiprocessors (SMs), while the full DICE system achieves performance comparable to the modeled Turing GPU baselines. DICE demonstrates that spatial pipeline execution can deliver substantial energy savings without sacrificing performance.

Index Terms—SIMT, coarse-grained reconfigurable array (CGRA), GPGPU architecture, spatial acceleration, memory coalescing

I. INTRODUCTION

The exponential growth in computational demands across domains such as machine learning, scientific computing, and high-performance computing has driven the widespread adop-

tion of massively parallel architectures. Among these, GPGPUs have emerged as the dominant platform for accelerating parallel workloads, extending beyond their original graphics rendering purpose through the SIMT programming model. However, their underlying SIMD execution introduces several inefficiencies that can be optimized. First, GPGPUs communicate intermediate data between instructions through centralized register files (RFs), resulting in frequent RF accesses, which contribute significantly to energy consumption. Second, although threads are grouped into warps to amortize front-end overheads (instruction fetch, decode, and schedule) [1], [26], these control stages remain energy-intensive due to their complexity and frequency of operation. Previous studies show that register file accesses and control logic combined consume up to 30% of total dynamic power [18], [25]. More recent studies [2], [19] further indicate that RF and control can account for over 40% of SM dynamic energy across modern GPU generations, including Pascal [33], Volta [34], and Turing [35], as well as more recent architectures such as A100 [36] and H100 [38].

These inefficiencies become increasingly problematic as we approach power density limits in modern process technologies. While specialized units like tensor cores [34] address specific workload domains, improving the energy efficiency of general-purpose SIMT execution remains critical for the broad spectrum of GPGPU applications that cannot leverage specialized units. To address these inefficiencies, we propose **DICE** (Dataflow Intelligent Compute Engine), which replaces the SIMD backend in contemporary GPGPUs with **minimal-overhead, statically scheduled, spatial-only coarse-grained reconfigurable arrays (CGRAs)**.

Unlike SIMD units that execute instructions in lockstep for a thread warp, DICE executes a massive number of threads on the spatial fabric in a pipelined fashion with an initiation interval (*II*) of one. Fig. 1 visualizes the differences between GPGPU SIMD lanes and the CGRA spatial pipeline computing model in DICE. On the CGRA fabric, intermediate values are passed directly between PEs via interconnects, greatly reducing RF accesses. DICE further reduces control overhead by using CTAs (Cooperative Thread Arrays) rather

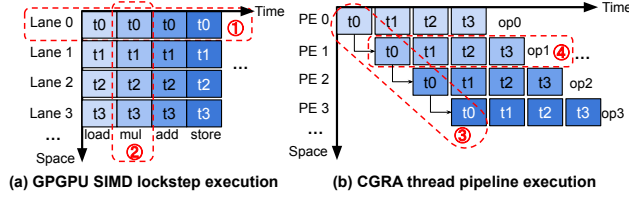


Fig. 1. GPGPU SIMD lockstep execution model vs. DICE CGRA thread pipeline execution model. ①: Each thread is bound to a single lane throughout execution. ②: All lanes execute the same instruction in lockstep. ③: Threads flow through different PEs in a pipelined manner. ④: Each PE is configured to perform a fixed operation for different threads.

than warps as the scheduling granularity, eliminating warp-level control logic overhead and amortizing remaining costs across hundreds of threads. Additionally, the pipeline execution scheme allows DICE to selectively dispatch only active threads to the CGRA, avoiding performance loss from masked SIMD execution under control divergence [12].

To support operations with runtime dynamism, such as variable-latency memory operations and data-dependent control flows in generic SIMT programs, DICE partitions control-dataflow graphs (CDFGs) into *p-graphs* where the producer-consumer pairs of these operations are strictly separated into different partitions (detailed in Section III). This decoupling ensures that each *p-graph* has a fixed, static schedule on CGRA, eliminating the need for elastic interconnects (e.g., FIFOs) between PEs. DICE uses a lightweight four-stage control pipeline to orchestrate the higher-level *p-graph* execution order (detailed in Section IV). To hide CGRA reconfiguration latency, DICE employs double-buffered configuration memory for rapid context switching.

Beyond the baseline architecture, DICE incorporates two critical optimizations. First, *p-graph* unrolling maps multiple instances of the same *p-graph* onto the CGRA simultaneously, enabling multiple threads to be dispatched per cycle. This optimization is effective for small *p-graphs* that would otherwise underutilize the CGRA fabric. Second, the temporal memory coalescing unit (TMCU) preserves memory efficiency in GPGPUs' memory system despite the pipeline thread execution scheme by buffering and merging requests from consecutive threads that access the same cache line, maintaining coalescing benefits without the lockstep execution requirement.

To summarize, this work makes the following contributions:

- We identify key inefficiencies in GPGPU SIMT implementation and propose **DICE**, a novel architecture that replaces the SIMD backend of GPGPUs with statically scheduled CGRAs to address these inefficiencies.
- We introduce *p-graph* partitioning that enables DICE to handle variable-latency memory operations and control flow without requiring additional hardware mechanisms such as elastic interconnects, dynamic scheduling logic, or flow control between PEs.
- We examine the bottlenecks in baseline DICE and develop two critical optimizations: *p-graph* unrolling to

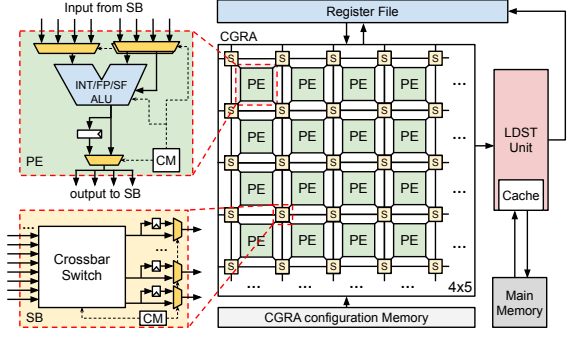


Fig. 2. CGRA architecture in DICE machine model. PEs and Switch Boxes (SBs) are scheduled at compile time. The register file serves as the interface between configurations: input operands flow from the register file to the CGRA fabric, while outputs are forwarded to the register file or the load/store Unit (LDST unit) for memory operations.

improve utilization and the temporal memory coalescing unit (TMCU) that preserves memory system efficiency despite the pipeline thread execution scheme.

- We evaluate DICE by extending Accel-sim [21] for cycle-accurate performance modeling, and develop a customized *p-graph* compiler. We further develop key components in RTL and synthesize them using Cadence toolchains with FreePDK45 [47]. We build the dynamic power model in AccelWattch [19] with Cadence Joules [6]. Evaluation on the Rodinia benchmark suite [7] shows that on average DICE reduces RF accesses by 68% on average. Moreover, DICE's CGRA processors (CPs) achieve a geometric mean of 1.77–1.90 $\times$ energy efficiency and 42.0%–45.9% average dynamic power reduction relative to the modeled NVIDIA Turing SM [35], while the full DICE system delivers performance comparable to the modeled Turing GPU baselines.

II. BACKGROUND

A. SIMT Programming Model

The SIMT model in modern GPGPUs offers a simple yet powerful abstraction for parallel programming, allowing developers to write sequential thread code while the hardware manages grouping and scheduling. Unlike SIMD, SIMT supports divergent control flow where threads may follow different branches and allows independent memory accesses without explicit gather or scatter instructions [26], [31]. GPGPUs implement SIMT through techniques such as warp-based SIMD backend execution, mask-stack for divergence handling, and memory coalescing [1]. DICE preserves the SIMT model to retain these software benefits while introducing a more efficient CGRA-based execution backend.

B. CGRA

CGRAs offer unique trade-offs in performance, energy efficiency, and flexibility compared to other hardware architectures such as CPUs, GPUs, ASICs, and FPGAs [16], [17], [22], [51]. A CGRA typically consists of an array of PEs

operating at word-level granularity, connected via a reconfigurable interconnect. Each PE performs arithmetic or logical operations and forwards data to the next PEs through the interconnect. Configurations for both PEs and interconnects are stored in a dedicated CGRA configuration memory (CM), which is programmed with bitstreams mapping computations onto the spatial fabric.

Program mapping depends on CGRA capabilities. In basic CGRAs without control-flow support, the program's control-dataflow graph (CDFG) is partitioned into basic blocks (BBs), each compiled into a dataflow graph (DFG) of pure data dependencies (control handled externally). Fig. 3 shows an example CDFG and a corresponding DFG for a simple program. More advanced CGRAs support direct mapping of control flow onto the fabric through mechanisms such as ϕ -nodes (select operations) for merging values from different paths [20], predicated execution [23], or dedicated control flow operations [14]. These approaches enable mapping larger CDFG portions, potentially the entire program, as a single configuration, improving performance by reducing reconfiguration overhead and enabling aggressive optimizations at the cost of additional hardware complexity and lower energy efficiency.

The CGRA design space is broad [27], [29], [42], but two dimensions are particularly relevant to DICE:

The first dimension distinguishes **spatial-only** CGRAs from **temporal-spatial** CGRAs: Spatial-only CGRAs [9], [13], [14] keep a single configuration active throughout execution, with each PE fixed to one operation. Temporal-spatial CGRAs [8], [20], [43], [49], [50] store multiple configurations and can switch between them every cycle, time-multiplexing resources to improve flexibility and utilization at the cost of extra silicon area and switching power.

The second dimension distinguishes **statically scheduled** CGRAs from **dynamically scheduled** CGRAs: Statically scheduled CGRAs [8], [9], [20], [49], [50] execute operations in a fixed order determined at compile time. Dynamically scheduled CGRAs fire operations when operands are ready, using flow-control (valid-ready or credit-based) [13], [14], [43] or tagged-token dataflow [3], [52]–[55] to distinguish multiple in-flight instances. Dynamic scheduling tolerates latency variability and irregular control/data patterns, but requires extra hardware for tagging, matching, and buffering.

A recent study [56] quantifies the area overhead of other CGRA styles relative to a statically scheduled, spatial-only CGRA. Temporal-spatial CGRAs incur a $1.6\times$ area overhead (+160%), while dynamically scheduled CGRAs add $1.1\times$ overhead (+110%) with flow-control and $4.8\times$ overhead (+480%) with tagged-token dataflow.¹

III. *p-graph* AND DICE MACHINE MODEL

To minimize hardware overhead, **DICE employs the statically scheduled, spatial-only CGRAs** as illustrated in Fig. 2. PEs can support integer (INT), floating-point (FP), or special

¹The statically scheduled, spatial-only CGRA baseline in [56] is denoted as “Systolic,” while temporal-spatial CGRAs are denoted as “CGRA.”

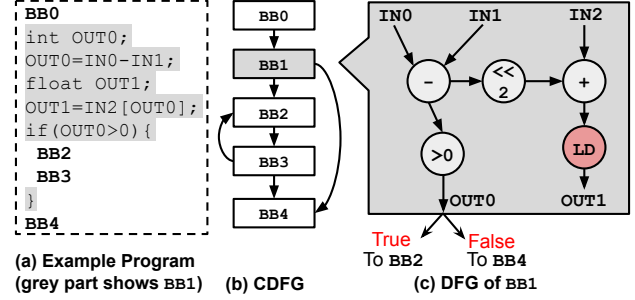


Fig. 3. Control-data flow graph example (LD: Load, BB: Basic block)

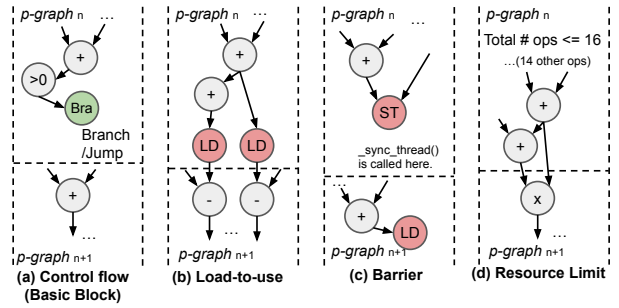


Fig. 4. *p-graphs* constraints

functional (SF) operations according to the arithmetic logic units (ALU) types, with a 1-bit control signal for predicated execution. The interconnect uses statically scheduled, wire-switched routing similar to the one in AHA [23].

A. *p-graph* formation

To support operations with runtime dynamism, including memory operations with unpredictable, variable latency, and data-dependent control flow operations, DICE partitions the CDFG into multiple subgraphs that are later compiled into different CGRA configuration bitstreams, separating producer-consumer pairs of these operations across different CGRA configurations and moving such dependence edges to CGRA boundaries for external handling. We call these partitioned subgraphs ***p-graphs***. As shown in Fig. 4, each *p-graph* satisfies the following constraints:

- 1) **Control flow constraint (Fig. 4(a)):** No control divergence (jump or branch instructions) within a *p-graph* (Same constraint in basic block). This constraint can be relaxed with predicated execution (Section IV-A3).
- 2) **Memory load constraint (Fig. 4(b)):** No memory load-to-use dependencies within a *p-graph*.
- 3) **Barrier constraint (Fig. 4(c)):** Synchronization barriers terminate a *p-graph*.
- 4) **Resource constraint (Fig. 4(d)):** resource usage must fit within CGRA capacity.

These constraints ensure that each *p-graph* is statically analyzable, has fixed latency, and is mappable within the spatial constraints of the DICE CGRA fabric.

TABLE I
p-graph METADATA COMPONENTS

Field	Value Type	Description
BITSTREAM_ADDR	32-bit address	Address of the corresponding CGRA configuration bitstream
BITSTREAM_LENGTH	8-bit value	Bitstream size in bytes
UNROLLING_FACTOR	2-bit value	Max thread unrolling factor (see Section IV-B1)
LAT	8-bit value	CGRA fabric latency for the <i>p-graph</i>
IN_REGS	34-bit bitmap	Input registers bitmap of the <i>p-graph</i>
OUT_REGS	34-bit bitmap	Direct output registers bitmap from CGRA fabric of the <i>p-graph</i>
LD_DEST_REGS	4×6 -bit value	Destination register indexes for memory loads (max request ports $\times$ index width)
NUM_STORES	3-bit value	Number of stores per thread of this <i>p-graph</i>
BRANCH_*	32-bit value	Branch/Jump metadata associated with this <i>p-graph</i>
BARRIER	1-bit bool	Barrier indicator, all previous blocks must retire before executing this <i>p-graph</i>
PARAMETER_LOAD	1-bit bool	Special <i>p-graph</i> for loading kernel parameters (see Section IV)

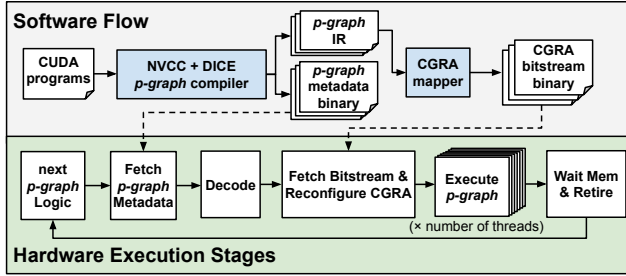


Fig. 5. DICE software flow and hardware execution stages

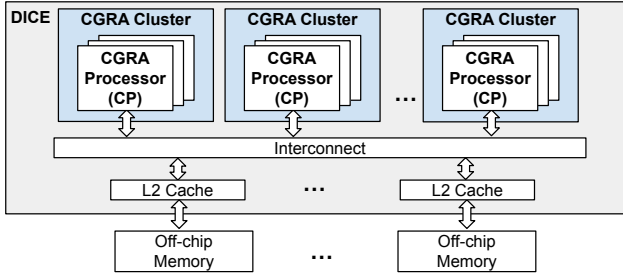


Fig. 6. DICE top-level organization. CGRA Processors (CPs) are grouped into clusters, which connect to the submemory system through a shared interconnect.

B. DICE Machine Model

DICE employs the register file for inter-*p-graph* data communication. To handle memory operations at the boundary of each *p-graph*, DICE employs a dedicated load/store unit (LDST Unit) placed alongside the CGRA as illustrated in Fig. 2. Operands are read from the register file into the CGRA, and CGRA outputs are either directly written back for subsequent *p-graphs* or routed to the LDST Unit for memory accesses. The LDST Unit fetches data from the memory system into the register file, completing the execution loop.

Fig. 5 illustrates both the software flow and hardware execution model of DICE across multiple *p-graphs*.

Software flow: Starting from programs written in CUDA [31], NVCC [40] compiles the high-level code into

PTX [39], an assembly-like intermediate representation (IR). The DICE compiler then: (1) constructs CDFGs from PTX IR, (2) partitions them into *p-graphs*, and (3) generates *p-graph* IR with associated metadata. Table I summarizes the metadata fields used to describe each *p-graph*. Like in conventional CPU/GPGPU instruction set architectures (ISAs), inputs and outputs are encoded as register numbers. The *p-graph* IR is then passed to a CGRA mapper, which performs placement and routing on the CGRA fabric and generates the final CGRA configuration bitstream.

Hardware execution stages: The execution stages across *p-graphs* resemble traditional von Neumann-style control pipelines with instructions. As illustrated in Fig. 5, the execution flow begins with (1) the next *p-graph* logic selecting the subsequent configuration. (2) The corresponding metadata is fetched and (3) decoded to identify the bitstream address and configure CGRA peripherals. (4) This CGRA bitstream is subsequently retrieved and loaded into the CGRA configuration memory. (5) Once reconfiguration completes, threads are dispatched to the CGRA to execute the current *p-graph* in a pipelined fashion with $II = 1$ as shown in Fig. 1. (6) Following CGRA computation completion, the system waits for outstanding memory operations before retiring the current *p-graph* and proceeding to the next. This cycle continues until all kernel *p-graphs* have executed on all threads.

IV. DICE ARCHITECTURE

As shown in Fig. 6, DICE follows a hierarchical organization similar to contemporary GPGPUs [1]: compute resources are grouped into CGRA Clusters (CC), each containing multiple CGRA Processors (CPs), with clusters connected to main memory via the system interconnect.

When a kernel launches, each CTA is assigned to a CP. Unlike GPGPUs that schedule at warp granularity, DICE operates at the CTA level: when the CGRA is reconfigured for a new *p-graph*, all threads from the entire CTA are dispatched sequentially to the CGRA in a pipeline manner. This approach eliminates the need for warp-level control logic and amortizes the overhead of *p-graph* metadata/bitstream fetching and CGRA reconfiguration across hundreds of threads, further improving energy efficiency. At runtime, when a CTA is

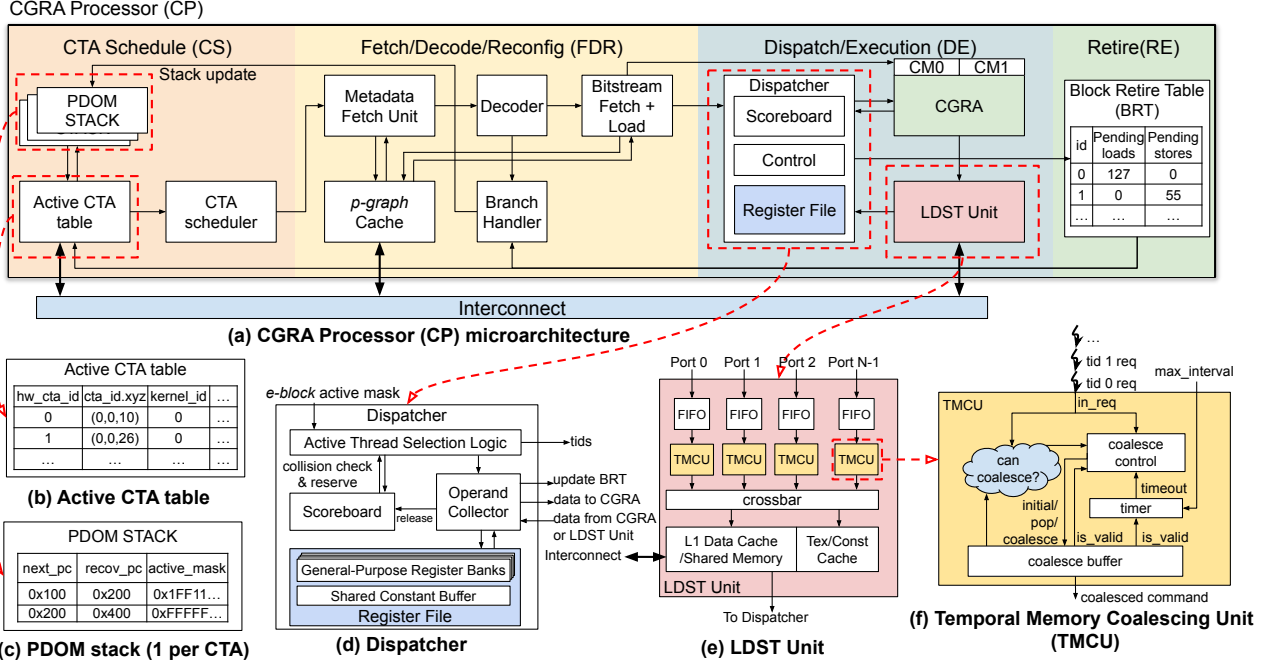


Fig. 7. CGRA Processor (CP) microarchitecture

scheduled to execute a specific p -graph, DICE instantiates a corresponding execution block, referred to as an e -block. An e -block is a dynamic runtime entity that encapsulates the execution of a p -graph for currently active threads in a given CTA. Conceptually, p -graphs and e -blocks are analogous to static and dynamic instructions, respectively, in conventional CPU/GPGPU terminologies.

The key design goal of the CP is to maximize CGRA utilization through efficient pipelining and organization.

A. CGRA Processor (CP) Baseline Microarchitecture

Fig. 7(a) illustrates the four pipeline stages of a CP in DICE: CTA schedule (CS), Fetch/Decode/Reconfig (FDR), Dispatch/Execution (DE), and Retire (RE). Fig. 8 presents an example execution timeline involving 2 CTAs assigned to a single CP, where the kernel is partitioned into 3 p -graphs. Blocks in the same color and pattern represent the same e -block passing through different stages.

1) **CTA Schedule (CS):** This stage selects the next CTA and forms an e -block for the next pipeline stages. The **Active CTA Table** (Fig. 7(b)) stores CTAs assigned by the kernel driver, along with their associated kernel metadata. Each active CTA is linked with a dedicated **Post-Dominator (PDOM) Stack** (Fig. 7(c)) similar to ones used in NVIDIA Fermi GPGPUs [1], [12] to track control divergence. The PDOM stack can handle both nested control flow and skipped computation entirely by stacking the next program counter (PC), reconvergence PC, and the active thread mask of each divergent path.

Scheduling Strategy: To maximize metadata and bitstream reuse to reduce overhead, the **CTA Scheduler** prioritizes CTAs

whose next PC matches that of recently dispatched e -blocks, while applying a round-robin policy among otherwise active CTAs. Once a CTA is selected, a new e -block is instantiated with the CTA's metadata and the next p -graph metadata PC and then forwarded to the next stage.

2) **Fetch/Decode/Reconfig (FDR):** In this stage, the **Metadata Fetch Unit** fetches the metadata for the corresponding p -graph from **p -graph Cache** (analogous to GPGPU's instruction cache), which is then decoded by the **Decoder**. The **Branch Handler** processes control flow information embedded in the p -graph metadata and determines whether the next p -graph of this CTA can be speculatively identified before this e -block completes execution. Then it updates the PDOM stack of the corresponding CTA accordingly.

Branch Prediction Strategy: The branch handler employs a backward-taken forward-not-taken (BTNT) prediction scheme. In Fig. 3, both **BB1** and **BB3** predict **BB2** as their successor. This predictor requires minimal hardware yet performs well when both paths contain active threads, allowing the CS and FDR stages to proceed without stalls. However, the e -block must wait before entering the DE stage until thread activeness is resolved; mispredicted e -blocks are discarded.

Handling Synchronization Barriers: When metadata indicates a synchronization barrier, the FDR stage must ensure all prior operations complete before proceeding to the DE stage. Unlike GPGPUs that track individual warp arrivals at barriers, DICE's CTA-level execution eliminates this complexity. The FDR stage simply waits for signals from the RE stage confirming that all prior e -blocks from the CTA have completed memory operations.

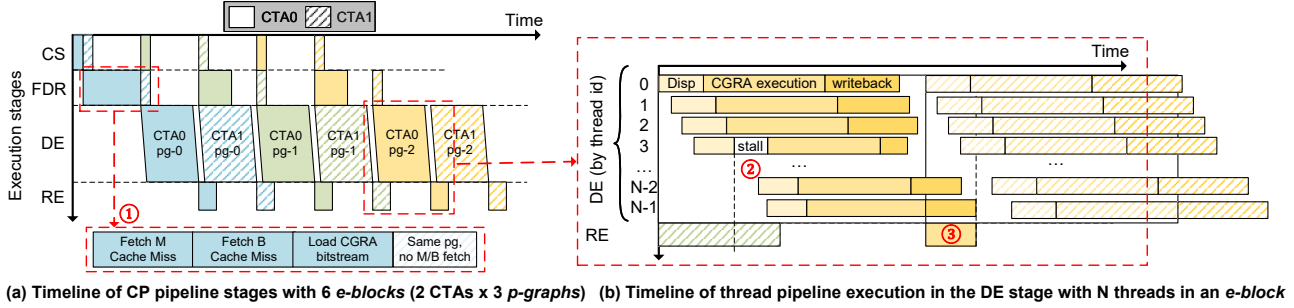


Fig. 8. CGRA Processor (CP) pipeline execution example (M: metadata, B: CGRA bitstream, pg: *p-graph*, Disp: dispatch). ①: The CTA scheduler selects CTA1 after CTA0 as the next *e-block* to run due to the same *p-graph* as the previous one. The FDR stage of *pg-0* for CTA1 is much shorter since metadata and bitstream from the previous fetch can be reused. ②: Dispatch stalls when scoreboard detects data dependency on pending memory writeback. ③: As soon as all threads complete CGRA execution, the *e-block* enters the RE stage and retires once all memory operations finish.

Double-buffered CGRA Configuration Memory: After decoding the address of the bitstream, the **Bitstream Fetch and Load** unit checks whether the required CGRA configuration bitstream is already resident in either one of two configuration memories (CM0/CM1, shown in Fig. 7(a)). Missing bitstreams are fetched from the *p-graph* cache and loaded into the inactive configuration memory, enabling reconfiguration without interrupting ongoing CGRA execution.

An *e-block* is considered ready to proceed to the next stage once all the following conditions are met: metadata decoding is complete, the CGRA bitstream is loaded, active thread information is resolved, and any barrier requirements (all prior *e-blocks* of the CTA completed and retired) are satisfied.

3) **Dispatch/Execution (DE):** This stage executes threads on the CGRA fabric through three main components: the **Dispatcher**, the **CGRA**, and the **LDST Unit**.

Selective Dispatch: The Dispatcher (Fig. 7(d)) orchestrates thread sequencing, manages operands, and tracks memory requests through four subcomponents: **Active Thread Selection Logic**, **Scoreboard**, **Register File**, and the **Operand Collector**. The Active Thread Selection Logic extracts the sequence of active threads from the *e-block* active mask and outputs them in ascending thread ID (tid) order. In this process, DICE skips inactive threads entirely, avoiding performance loss in masked SIMD execution.

Conflict-free Register File and Management: The register file (RF) stores intermediate values exchanged across *e-blocks*. It includes multiple **General-Purpose Register Banks** storing private registers of each thread, and a **Shared Constant Buffer** for constant values like function arguments that are shared across all threads in the same kernel. To maintain $II = 1$, the RF employs a banking scheme where registers with different indices map to separate banks, while same-index registers across threads share a bank. This organization enables conflict-free access to all of a thread's registers simultaneously, while avoiding energy waste from fetching unnecessary registers with fewer banks. The Operand Collector coordinates data movement—fetching operands, managing writeback, and tracking memory operations—while the Scoreboard prevents data hazards by verifying operand availability before actually

dispatching a thread.

Predicated Execution and Writeback: CGRA outputs are routed to both the RF (via the Operand Collector) and the **LDST Unit** (Fig. 7(e)). The LDST Unit in baseline DICE resembles its GPGPU counterpart with a fixed number of ports, except memory requests from different threads arrive sequentially through the same port over pipeline cycles, buffered by a FIFO. (The optimized DICE design incorporates a temporal memory coalescing unit (TMCU), described in Section IV-B2.) As described in Section III, each PE has a 1-bit control signal input for predicated execution. These control signals also serve as CGRA output controls: they enable/disable register writeback and mark memory requests as valid/invalid. This predication support allows DICE to translate control flow into predicated execution, enabling the merging of *p-graphs* that were originally separated due to control divergence. Instead of being separated into different *p-graphs* for divergent paths, both paths execute within a single *p-graph* with their operations selectively enabled by predication bits, improving utilization and reducing reconfiguration overhead.

Stall Conditions: The dispatcher stalls when: (1) the scoreboard detects the current dispatching thread's input registers are still awaiting memory loads, (2) any FIFO in the LDST Unit runs out of credit (reaches the capacity), or (3) no available threads to dispatch (Idle state). The idle state can occur when no ready *e-block* is available in the previous stage, or when all threads of the current *e-block* have been dispatched, but CGRA computation is still in progress.

Pipeline Fill/Drain Overhead: The CGRA incurs bubbles during pipeline fill and drain, adding an extra latency of p cycles to the execution of a *p-graph* with latency p . For a CTA with t threads, the total execution time is therefore $t + p$ cycles when dispatch is not stalled. Since each CTA typically contains hundreds of threads, the relative fill/drain bubble overhead of p/t on overall performance is small.

4) **Retire (RE):** To avoid occupying CGRA compute resources after all threads in an *e-block* have completed CGRA execution but are still waiting on memory responses, DICE employs the RE stage with the **Block Retire Table (BRT)**. The BRT, illustrated in Fig. 7, can hold multiple *e-blocks* and tracks

Algorithm 1 Temporal Memory Coalescing Logic

```
function OUTPUTCOMMAND(in_req)           // Run every cycle
  if coalesce_buffer.is_valid() then
    timer.decrement()
  end if
  if timer.timeout() then
    coalesce_buffer.pop()                 // Pop the req in buffer
    timer.reset(max_interval)
  end if
  if in_req.is_valid() then               // Process new request
    if not coalesce_buffer.is_valid() then
      coalesce_buffer.initial(in_req)
    else if coalesce_buffer.can_coalesce(in_req) then
      coalesce_buffer.coalesce(in_req)
    else
      coalesce_buffer.pop()               // Pop to cache
      timer.reset(max_interval)
      coalesce_buffer.initial(in_req)
    end if
  end if
end function
```

the status of their outstanding memory operations. An *e-block* transitions from the DE stage to the RE stage once all of its threads have been dispatched and have completed computation operations on the CGRA fabric, as shown in Fig. 8(b)③. While in the BRT, *e-blocks* are monitored for completion of pending memory requests. Once all outstanding memory operations have completed for an *e-block*, the BRT retires the *e-block* by removing it from the BRT and updates the remaining *e-block* status to Branch Handler and Active CTA table. The RE stage allows the CGRA fabric to continue executing subsequent *e-blocks* without stalling on memory latency, thereby improving throughput and overall resource utilization.

B. Optimizations upon Baseline DICE

1) *Thread Unrolling for Enhanced Utilization*: While predicted execution enables the merging of divergent paths into larger *p-graphs*, some remain inherently small due to other constraints, leading to CGRA underutilization. DICE addresses this through thread unrolling similar to loop unrolling in DySER [15].

Key challenges arise when unrolling threads for CGRA execution: (1) maintaining $II = 1$ with multiple simultaneous threads, since unrolling by $2\times$ with $II = 2$ provides no benefit over single-thread dispatch with $II = 1$; (2) avoiding complex runtime hardware mechanisms and corresponding metadata; and (3) keeping compiler complexity manageable while enabling optimization flexibility.

DICE employs hardware/software co-design to address these challenges. On the hardware side, DICE adopts a hardware swizzled register bank mapping [41], where each thread's private registers are shifted by one bank relative to neighboring threads. Assuming the total number of logical registers is N_r , the number of physical banks is also equal to N_r as stated in section IV-A3. Physical thread T 's register R_r^T in a CP is mapped to bank $((r+T) \bmod N_r)$, ensuring adjacent threads access different banks. The dispatcher supports multi-

ple lanes but restricts co-dispatched threads to fixed intervals of K ; that is, only threads $(T, T+K, T+2K, \dots)$ can be dispatched together. The maximum simultaneous threads that can be dispatched is N_{Tmax} .

The DICE compiler is aware of the hardware's swizzling policy and uses this information to statically determine the maximum safe unrolling factor for each individual *p-graph*. Given a set of input registers $(R_{a1}, R_{a2}, \dots, R_{ap-1})$ for a *p-graph*, the compiler finds the maximum unrolling factor that avoids bank conflicts and respects hardware resource limits.

When bank conflicts limit unrolling, the compiler is able to adjust register allocation at compile time to get higher unrolling factors. In the DICE model used for evaluation, $N_r = 32$, $N_{Tmax} = 4$, with $K = 8$ for $4\times$ unrolling and $K = 16$ for $2\times$ unrolling ($3\times$ unsupported).

2) *Temporal Memory Coalescing Unit*: Memory coalescing reduces pressure on the memory system by combining multiple narrow memory requests into fewer, wider memory transactions, improving memory bandwidth utilization, lowering access latency, and reducing energy consumption. In GPGPUs, threads within a warp issue memory requests simultaneously, enabling the coalescing unit to identify and merge accesses in parallel. GPGPU programs are typically optimized to exploit this coalescing behavior.

However, this coalescing strategy does not directly apply to DICE, where memory requests arrive sequentially over multiple cycles through a shared port, lacking the simultaneous visibility that enables GPGPU-style coalescing.

While Miss Status Holding Registers (MSHRs) [24] track outstanding misses to support non-blocking caches, they are not a coalescing mechanism: they retain per-miss state after a request is issued, but do not opportunistically merge neighboring requests into a wider transaction.

To preserve coalescing benefits in pipelined execution, we introduce the Temporal Memory Coalescing Unit (TMCU). As shown in Fig. 7(f) and Algorithm 1, the TMCU maintains a coalescing buffer for gathering and merging compatible memory requests. When a new memory request arrives, the TMCU evaluates whether it can be coalesced with the buffered transaction based on request type and address alignment (can_coalesce() function in Algorithm 1). If the request can be coalesced, it is merged into the command in the buffer; otherwise, the existing buffered command is popped and sent to L1 caches, after which the new request becomes the base command in the coalesce buffer.

To prevent commands from remaining in the coalescing buffer indefinitely due to a lack of subsequent requests, the TMCU includes a timeout mechanism. A timer tracks how long a coalesced command has remained in the buffer, and once it exceeds a preset max interval, the command is flushed to the memory system to ensure forward progress. Under typical access patterns where consecutive threads access contiguous memory, the TMCU achieves coalescing equivalent to traditional GPGPU coalescing units. In rare corner cases (e.g., when requests from threads 0 and 2 are able to coalesce but thread 1 is not), the TMCU may fail to merge them due to its

TABLE II
DICE AND NVIDIA TURING GPGPU (RTX2060S) CONFIGURATIONS IN EVALUATION.

Category	Component	RTX2060S [35]	DICE (This work)
Organization	Compute Clusters	34 SMs	34 CGRA Clusters (CC)
	Core Partition	4 subcores/SM	4 CPs/Cluster
	Max threads	1024/SM	2048 [#] /Cluster
Functional Units	ALUs	16 CUDA Cores / subcore [*] 2176 CUDA Cores total	16 PEs [†] / CP 2176 PEs total
	Register File	256 KB/SM 8.5 MB total	32 Banks/CP, 256 KB/Clusters 8.5 MB total
Memory	L1 Cache / Shared Memory	96 KB/SM, 3.2 MB total	96 KB/Cluster, 3.2 MB total
	L2 Cache	4096 KB total (16-way, 64 sets)	
	DRAM	GDDR6 DRAM, 8 channels	
Frequency	Core, Interconnect, L2 Cache	1470 MHz	
	DRAM	1750 MHz	

[#]DICE allocates 32 private 32-registers per thread. With the same total RF capacity, DICE can keep more thread contexts resident because spatial execution reduces register pressure. ^{*}In RTX2060S, each CUDA core has separate INT and FP units that can execute simultaneously. [†]In DICE, each PE can perform either one INT or FP operation per cycle, but not both concurrently.

in-order and temporal processing constraints. However, such scenarios are infrequent in well-optimized GPGPU programs.

C. DICE Scalability

DICE exhibits two forms of scalability, similar to other spatial architectures [44]: *scale-up* and *scale-out*.

Scale-up (Larger CGRAs). Increasing the CGRA size enables larger *p-graphs* to be mapped onto the fabric, allowing more operations to execute spatially and improving performance. A larger fabric can also eliminate certain resource-induced partitioning boundaries, reducing intermediate register file storage previously required due to capacity constraints. Furthermore, *p-graphs* that are not resource-bound can exploit additional spatial capacity through further unrolling to maintain high utilization. However, for small *p-graphs* that have already reached the maximum supported unrolling factor, increasing CGRA size alone may lead to lower PE utilization.

Scale-out (More CPs). Scale-out increases the number of CGRA clusters while keeping each CP microarchitecture unchanged. This approach mirrors how GPU families scale by varying the number of SM per die (e.g., RTX2060S (34 SMs), Quadro RTX5000 (48 SMs), and Quadro RTX6000 (72 SMs) in NVIDIA Turing GPGPUs family [35]) while preserving the SM design. Since each CP is unchanged, scale-out increases peak compute capacity and typically improves throughput in the same way that adding more SMs does in GPUs, subject to workload parallelism and memory-system limits.

V. EXPERIMENTAL METHODOLOGY

A. Basic Power, Performance, Area Evaluation Methodology

Hardware modeling: We model DICE by extending the Accel-sim [21] cycle-accurate GPGPU simulator. We further develop RTL for DICE-specific components and synthesize them using the open-source FreePDK45 process design kit [47] with the Cadence toolchain. We also use CACTI [30] to accurately model memory area and timing characteristics,

TABLE III
EVALUATED KERNELS AND LAUNCH CONFIGURATIONS.

Benchmark: Kernel	# <i>p-g</i>	B	G	T
NN: euclid	4	256	2048	524288
BFS-1: kernel	10	512	128	65536
BFS-2: kernel2	4	512	128	65536
BPNN-1: layerforward	10	256	256	65536
BPNN-2: adjust_weights	7	256	256	65536
SC: compute_cost	12	512	128	65536
GE-1: Fan1	5	512	1	512
GE-2: Fan2	6	256	169	43264
HS: calculate_temp	13	256	1849	473344
PF: dynproc_kernel	8	256	544	139264

#*p-g*: Number of *p-graphs*; B: Block/CTA size (flattened); G: Block grid size (flattened); T: Total threads ($T = B \times G$).

and all results are subsequently scaled to 12nm FinFET technology node with scaling factors in [46] to compare with the RTX2060S baseline.

To estimate the dynamic power and energy consumption of DICE, we extend the AccelWattch [19] power modeling framework to construct the dynamic power model of DICE. To ensure that the reported energy differences arise from architectural changes rather than circuit-level modeling discrepancies, we apply a consistent methodology to both designs. We use AccelWattch’s per-access energy for hardware components that are the same across DICE and the baseline GPU (e.g., L1 cache, Shared Memory, Compute). SRAM blocks introduced by DICE are modeled with CACTI. As for DICE-specific control logic, we estimate energy consumption with RTL implementation and realistic activity trace-based analyses using Cadence Joules [6].

Compilers: We develop a customized DICE *p-graph* compiler, which translates PTX [39] code compiled from NVCC [40] to CDFGs and partitions CDFGs into *p-graphs*. The compiler incorporates an optimization pass to generate performance-optimized *p-graphs*. While our compiler handles

most kernels automatically, we apply manual optimizations in some cases to ensure architecture evaluation is not limited by compiler maturity.

Baseline: We compare DICE against the NVIDIA Turing GPGPU RTX2060 Super [35] (RTX2060S) in terms of overall performance and dynamic power and energy consumption at the core level (DICE CPs vs. RTX2060S SMs). To our knowledge, the RTX2060S is currently the most advanced model with detailed dynamic power modeling support in AccelWattch [19]. To ensure a fair comparison, DICE is configured to match the RTX2060S in both compute resource count and total cache/SRAM capacity as shown in Table II. DICE CPs use 4×5 CGRAs composed of 16 PEs and 4 Special Function Units (SFUs). In addition to this cross-architecture comparison, we evaluate internal DICE baselines in which thread unrolling and memory coalescing are selectively disabled or enabled in isolation, allowing us to quantify the individual and combined contributions of these techniques to performance. Specifically:

- **DICE-naïve:** Both *p-graph* unrolling and TMCU are off.
- **DICE-naïve+unroll:** Only unrolling is enabled while TMCUs are disabled. DICE supports unrolling factors equal to 2 and 4.
- **DICE-naïve+TMCU:** Only TMCUs are enabled with *max_interval* equals 8, which matches the 32-byte L1 data cache sector size with 4-byte data access. Unrolling is disabled.
- **DICE:** Both optimizations are applied.

Benchmarks: We evaluate DICE using diverse benchmarks from the Rodinia benchmark suite [7]. Table III summarizes the selected benchmarks and their kernel configurations.

B. Scalability Evaluation Methodology

1) *Scale-up (Larger CGRAs):* To study CGRA scale-up, we compare the baseline 16-PE DICE with DICE-U, a 32-PE design that doubles the CGRA size per CP. We scale RF, L1, and supported threads per CP proportionally. To keep per-cluster compute and memory constant, we halve the number of CPs per cluster, isolating the performance–RF-access trade-off of larger CGRAs (Table IV).

2) *Scale-out (More CPs):* To evaluate system-level scaling, we model larger Turing-class GPUs in Accel-sim by increasing the number of SMs to match the Quadro RTX5000 (48 SMs) and Quadro RTX6000 (72 SMs) [35]. DICE is correspondingly scaled by increasing the number of CGRA clusters to match the SM counts (referred to as **DICE-O48** and **DICE-O72**). Detailed hardware configurations are summarized in Table V. For selected kernels, we also increase problem sizes to ensure full SM occupancy.

VI. EVALUATION RESULTS

A. RF Accesses

Fig. 9 shows the normalized RF access counts in DICE relative to RTX2060S and demonstrates that, for the same program, DICE requires only 32% of the RF accesses compared with RTX2060S on average. This 68% reduction is primarily

TABLE IV
SCALE-UP DEVICE CONFIGURATION: DICE VS. DICE-U
CONFIGURATIONS (SAME TOTAL COMPUTE RESOURCE COUNTS).

Component	DICE	DICE-U
CPs/Cluster	4 CPs/Cluster	2 CPs/Cluster
ALUs	16 PEs/CP, 64/Cluster	32 PEs/CP, 64/Cluster
SFUs	4/CP, 16/Cluster	8/CP, 16/Cluster
LDST Units	4/CP, 16/Cluster	8/CP, 16/Cluster

Note: Parameters not shown are identical to those in Table II in SM.

TABLE V
SCALE-OUT DEVICE CONFIGURATIONS.

Component	Quadro RTX5000	DICE-O48	Quadro RTX6000	DICE-O72
# SM or CC	48	48	72	72
L2 Cache DRAM	4096 KB total 8 channels		6144 KB total 12 channels	

Note: RTX5000/6000 share the same Turing SM as RTX2060S [35]; therefore, all SM-level parameters are identical to those in Table II.

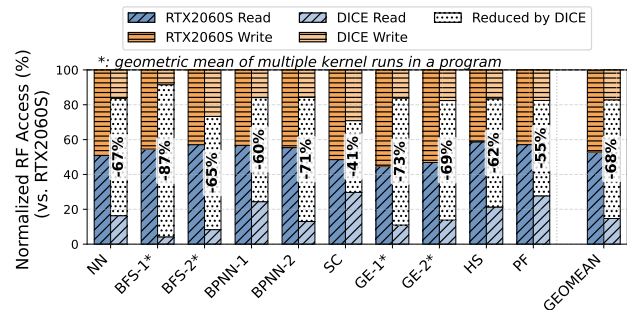


Fig. 9. Normalized RF accesses (%) (DICE vs. RTX2060S)

attributed to the spatial computing nature of the CGRA, where intermediate data is directly forwarded across wires. Note that thread unrolling and TMCU do not impact RF access counts.

B. Performance

Fig. 10 presents the performance comparison between RTX2060S baseline, DICE-naïve, DICE-naïve+unroll, DICE-naïve+TMCU, and DICE as described in Section V-A. Fig. 11 presents a detailed breakdown of cycle distribution and functional unit utilization for both RTX2060S and DICE architectures.

1) *DICE vs. RTX2060S:* Overall, DICE achieves a geometric mean speedup of $1.16\times$ over RTX2060S, demonstrating that spatial thread-pipeline execution on CGRAs can match conventional SIMD architectures despite fundamental design differences. This performance advantage stems from several architectural trade-offs that we analyze below:

Reduced Data Movement Instructions: The spatial execution model eliminates certain data-movement instructions

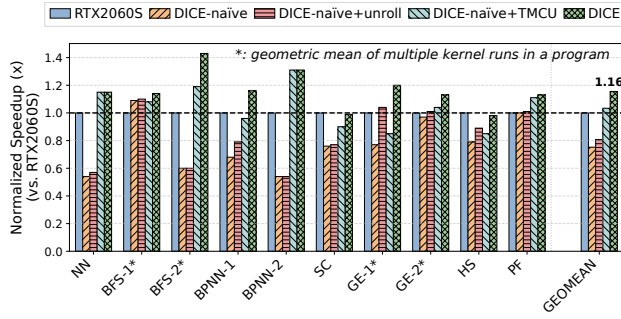
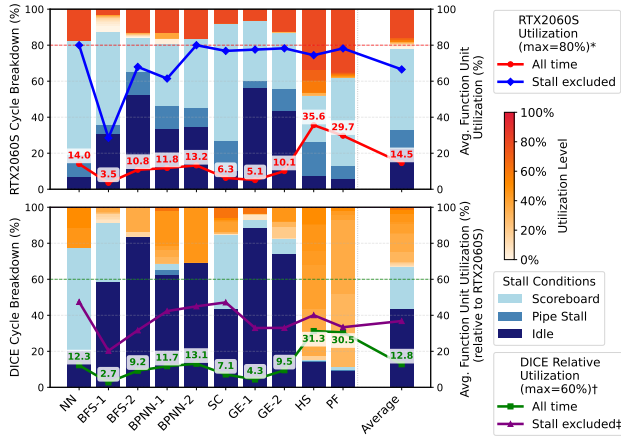


Fig. 10. Speedup (x) (DICE variants vs. RTX2060S)



*Maximum GPU utilization is 80% due to dispatcher constraints: each RTX2060S SM contains 160 functional units (64 INT, 64 FP, 16 LD/ST, 16 SFU) but can only dispatch up to 128 threads per cycle.

†DICE relative utilization is capped at 60%: each CGRA cluster has 96 functional units (4 CPs $\times$ 24 functional units) compared to RTX2060S's 160 units per SM.

‡Also represents the average p -graph size including memory operations.

Fig. 11. Cycle breakdown and average functional unit utilization. Top: RTX2060S. Bottom: DICE.

common in GPGPU workloads—such as MOV (register/value-to-register moves) and S2R (special register moves)—by forwarding values directly between PEs over the CGRA interconnect. This reduces static instruction count by up to 17.8% in kernels with relatively low per-thread instruction counts (NN, BFS-2, GE-1). Additionally, branch operations are handled in the control pipeline, reducing actual instruction counts per thread. The combined effects explain why DICE can remain performance-competitive with, and in some kernels outperform RTX2060S with lower overall utilization in Fig. 11.

Selective Dispatch: Selective dispatch avoids performance loss from GPGPU's mask-based SIMD backend by only dispatching active threads, as evidenced by RTX2060S's lower stall-excluded utilization in divergence-frequent kernels (BFS-1, BFS-2, BPNN-1).

Improved Memory Latency Hiding: DICE experiences fewer scoreboard stalls (Fig. 11), reflecting more effective

memory latency hiding. This benefit arises from two factors. First, DICE supports 2048 threads per cluster, which doubles the 1024 threads per SM in RTX2060S while maintaining the same register file capacity as RTX2060S (256 KB/SM), thanks to the reduced per-thread register demand enabled by spatial execution. Second, p -graph execution naturally spreads memory requests more evenly over time, reducing memory congestion and improving overlap with computation.

2) **DICE Utilization Analysis:** Cycle breakdowns in Fig. 11 show that DICE sustains more active cycles than RTX2060S but at a lower average active utilization, reflecting smaller p -graph sizes as indicated by its stall-excluded utilization curve. Utilization varies considerably across kernels. Kernels with regular, compute-heavy codepaths (NN, GE-2) achieve high utilization. Compute-intensive kernels with frequent shared-memory accesses (BPNN-1, BPNN-2, HS, PF) see reduced p -graph sizes because shared-memory load latencies cannot be predicted statically due to potential bank conflicts and LDST Unit stalls; frequent synchronization barriers further shorten p -graphs. We leave optimizations of shared memory access with CGRA pipeline execution to future work. Kernels such as BFS-1, BFS-2, SC, and GE-1 exhibit the lowest utilization due to irregular, data-dependent memory accesses, frequent control divergence, and low computational intensity.

The overall results show that DICE's architectural advantages yield higher temporal utilization, which offsets its relatively lower spatial utilization and delivers marginally better performance than the modeled RTX2060S.

3) Optimization Effectiveness:

a) **Impact of Thread Unrolling:** BPNN-1, GE-1, and HS achieve 1.24–1.35 $\times$ speedup over DICE-naïve, indicating small p -graph sizes without unrolling. Other kernels show little benefit, as their performance is bottlenecked by memory and interconnect bandwidth, or the p -graphs are already large enough that unrolling is not needed.

b) **Impact of Temporal Memory Coalescing:** NN, BFS-2, BPNN-1, and BPNN-2 achieve 1.41–2.45 $\times$ speedup with TMCU only over DICE-naïve, showing prior memory/interconnect bandwidth limitations. Unlike MSHRs, TMCU merges requests before cache access, producing a single cache access and response that better utilizes cache process bandwidth. Additionally, with the write-through cache policy used in both DICE and RTX2060S, all write requests are immediately forwarded to the system interconnect, creating congestion. By merging writes before they enter the interconnect, TMCU substantially reduces traffic, alleviates contention, and improves effective memory bandwidth utilization.

c) **Combined Optimizations and Performance Saturation:** For kernels such as BFS-2, BPNN-1, SC, GE-1, GE-2, and HS, DICE significantly outperforms either unrolling or TMCU alone. Unrolling raises resource utilization but also increases memory request rates, shifting the bottleneck to memory and interconnect bandwidth. This synergy yields a 1.54 $\times$ speedup over the DICE-naïve. In contrast, other kernels show negligible to modest improvement from unrolling after TMCU, as memory bandwidth is already saturated. These

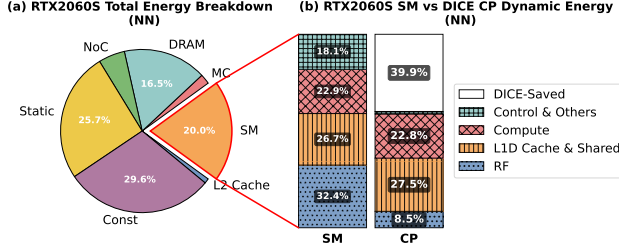


Fig. 12. Energy breakdown of NN benchmark in (a) RTX2060S system (b) RTX2060S SMs and DICE CPs (normalized)

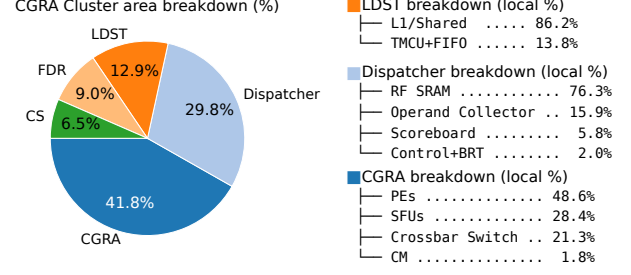
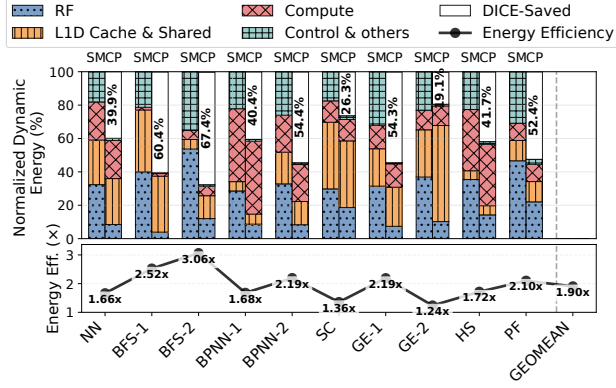
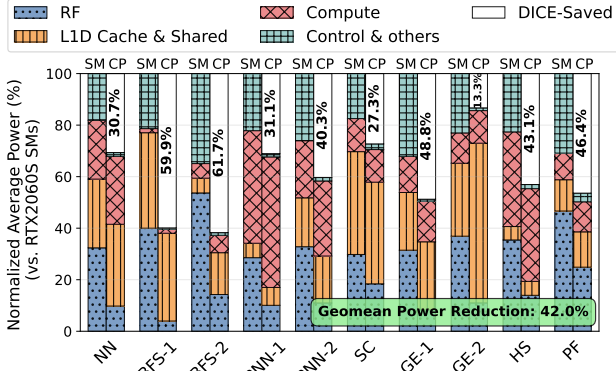


Fig. 14. DICE area breakdown



(a) Normalized energy breakdown and energy efficiency (x)



(b) Normalized average dynamic power consumption

Fig. 13. Energy & power efficiency (DICE CPs vs. RTX2060S SMs)

results indicate that DICE can both alleviate architectural inefficiencies and, in some cases, drive performance to the hardware's fundamental limits.

C. Energy Efficiency and Power

We evaluate energy efficiency and power consumption between DICE and RTX2060S. Fig. 12(a) presents the system-level energy distribution of RTX2060S on NN benchmark, breaking down consumption across Streaming Multiprocessors (SMs), Network-on-Chip (NoC), L2 caches, memory controllers (MCs), idle cores, off-chip DRAM, and auxiliary

circuits (CONST). The left bar chart in Fig. 12(b) further decomposes the SM dynamic energy into individual parts. Since DICE primarily replaces GPGPU SMs with CPs, and separating static energy in AccelWatch [19] is challenging, we focus on the dynamic energy consumed in SMs and CPs.

The right bar chart in Fig. 12(b) illustrates the normalized energy consumption breakdown of the DICE CP components relative to the RTX2060S SMs on NN benchmark. The white portion (39.9%) represents the total energy savings achieved by the CPs. These savings come from multiple factors. First, reduced RF accesses substantially lower overall energy use from 32.4% to 8.5%; Second, DICE's lightweight control pipeline reduces control logic energy from 18.1% to 1.3% by eliminating warp-level control logic and amortizing remaining control overhead across hundreds of threads.

L1 data cache and shared memory accesses also consume over 26.7% of SM dynamic energy. In DICE CPs, this energy is comparable to the RTX2060S SMs thanks to the effective coalescing provided by the TMCU, underscoring the importance of memory coalescing. Without it, the increased cache accesses would substantially raise energy consumption. However, even with the TMCU, CP's L1 energy is slightly higher than SMs in some cases, as the TMCU's effectiveness diminishes when coalescable requests arrive non-contiguously or when thread dispatching stalls.

Fig. 13(a) extends this analysis across all benchmarks. DICE CPs achieve a geometric mean of 1.90x energy efficiency compared to RTX2060S SMs. An exception occurs in GE-2, where DICE spends more energy on L1D cache and shared memory due to a load from the same address across all threads. This overhead could be eliminated by placing the data in the shared constant buffer reusable across all threads.

Fig. 13(b) further shows that DICE reduces average power consumption by 42.0%. This average power reduction closely tracks the energy savings because DICE's performance gains are modest (1.16x on average), meaning the energy efficiency improvements translate almost directly to power savings. This substantial power reduction is particularly valuable for thermally constrained environments and enables higher computational density within fixed power budgets.

D. Area Comparison

To estimate area, we implement the DICE-specific RTL

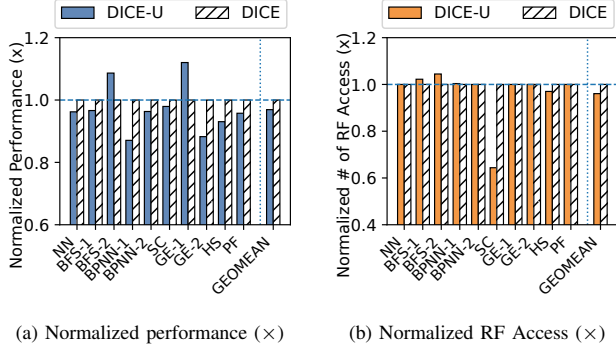


Fig. 15. Normalized performance and number of RF accesses (DICE-U vs. DICE)

components and synthesize them in FreePDK45 [47], while modeling SRAM/cache structures with CACTI [30]. The total area of a DICE CGRA cluster is 16.21 mm², with the detailed breakdown shown in Fig. 14. Because public area breakdowns for commercial GPUs are limited, we first provide a qualitative comparison against an RTX2060S SM, then give a conservative quantitative estimate based on die-shots.

Qualitative analysis: We divide DICE’s area into four categories. A_1 : GPU-matched components, including compute units and SRAM structures, whose total area should be broadly comparable to that of an RTX 2060 Super SM. DICE’s unified INT/FP datapath may further reduce PE area by up to 30% [4], though we do not include this benefit in the estimate below. A_2 : DICE-specific components, including the CGRA switches and configuration memory, account for 9.7% of a DICE CP’s area. A_3 : modified components, including the PDOM-stack, operand collector, scoreboard, control logic, and TMCU, account for 12.1% of a DICE CP’s area. A_4 : removed GPU components include the instruction buffer, warp scheduler, and warp-related control logic. The total core area can therefore be written as $A_{DICE} = A_1 + A_2 + A_{3,DICE} - A_{3,GPU} - A_4$. We assume $A_{3,DICE} \approx A_{3,GPU}$, as DICE modifies but does not fundamentally scale these structures, and conservatively set $A_4 = 0$. This yields $A_{GPU} \approx A_1 + A_{3,GPU}$, and thus the relative upper-bound area overhead can be expressed as $A_{DICE}/A_{GPU} - 1 = 10.7\%$.

Estimated quantitative comparison: We use die shots of the RTX 2060 Super [10] and GTX1660Ti [5], [11], both 12nm Turing GPUs, where the latter’s SM excludes Tensor Cores and RT cores. From these, we estimate an SM area of 5.44 mm² for the RTX2060S and 4.46 mm² for the GTX 1660Ti. In comparison, a DICE CGRA cluster occupies 2.92 mm² after conservative scaling to 12nm [46]. Although this is still an approximate comparison, it suggests that a DICE CGRA cluster is in the same general area range as, if not smaller than, a full GTX 1660Ti SM.

E. Scalability Evaluation

1) *Scale-up (Larger CGRAs):* As shown in Fig. 15, substituting larger CGRAs while keeping the total resource the

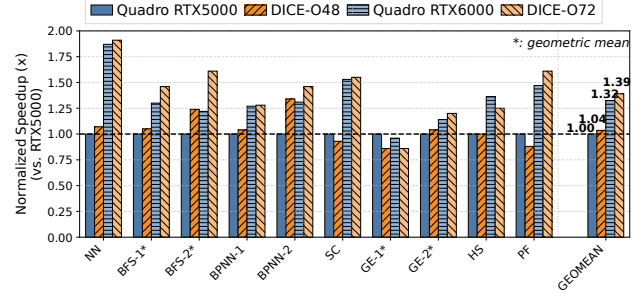


Fig. 16. Speedup (x) of DICE-O48, DICE-O72, and RTX6000 (vs. RTX5000).

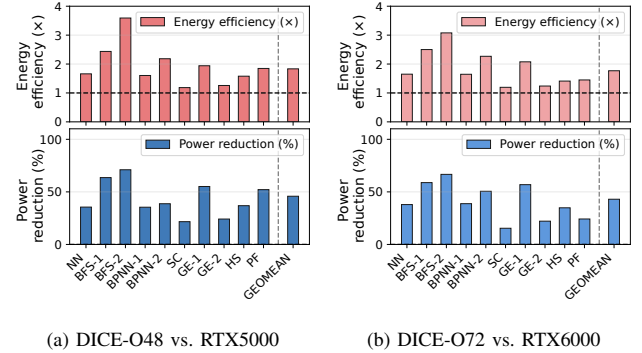


Fig. 17. Dynamic energy efficiency and power reduction (DICE CPs vs. Turing SMs)

same degrades the performance slightly ($0.97\times$) due to more underutilized PEs, as some workloads cannot fully exploit the enlarged spatial fabric. On the positive side, RF accesses decrease by approximately 3.8% overall, since larger CGRAs enable mapping of bigger *p-graphs*, particularly in compute-intensive kernels such as SC and HS. However, for control-divergent workloads such as BFS and GE-1, RF accesses slightly increase. This is because merged *p-graphs* introduce more predicated execution, causing registers of masked-off threads to be read even though their results are eventually discarded. Overall, scale-up is inherently limited. The exploitable *p-graph* size of most workloads is constrained by memory dependences and control divergence, and larger CGRAs increase compiler and mapping complexity. Thus, simply enlarging the CGRA is not a scalable path for performance improvement.

2) *Scale-out (More CPs):* Fig. 16 and Fig. 17 compare DICE against larger Turing GPUs (Quadro RTX5000 and RTX6000). The corresponding DICE variants proportionally scale the number of CPs to match the compute capacity of these GPUs. The results are consistent with our earlier evaluation on RTX2060S: DICE achieves performance comparable to the scaled Turing GPU baselines ($1.04\times$ – $1.05\times$), while delivering substantially better energy efficiency ($1.77\times$ – $1.83\times$) and significant average power reductions (43.0%–45.9%). Overall, these results demonstrate that DICE preserves the desirable scale-out property of GPUs. Performance scales

TABLE VI
DICE-UO vs. RTX3070 CONFIGURATION (1132 MHz)

Component	RTX3070 [37]	DICE-UO
# SM or CC	46	46
Core Partition	4 subcore/SM	4 CPs/Cluster
CUDA Cores	32*/subcore	32 PEs/CP
L1 Cache/Shared	128 KB/(SM or CC)	

Note: Parameters not listed here are identical to those in Table II at SM/Cluster level. *Half of the CUDA cores support FP32/INT32, while the other half support FP32 only.

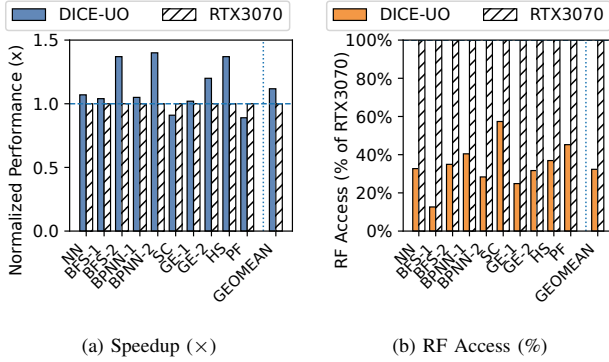


Fig. 18. Speedup (x) and RF Accesses Ratio (%) (DICE-UO vs. RTX3070)

effectively by increasing the number of CPs, while maintaining superior energy efficiency and power advantages.

3) *Performance Comparison Against Newer GPUs:* We additionally compare DICE against the RTX3070. Since each RTX3070 SM offers $2\times$ the FP32 throughput of a Turing SM, we scale DICE both up and out (**DICE-UO**) to match the RTX3070 compute resource count, as summarized in Table VI. As shown in Fig. 18, DICE maintains comparable average performance while reducing RF accesses by 68%, indicating that DICE’s efficiency carries over to newer GPU baselines.

VII. RELATED WORK

CGRA Memory Handling: Prior CGRAs handle variable memory latency through three main approaches. Scratchpad-based designs (HyCube [20], OpenCGRAs [50], AHA [23]) restrict memory operations to fixed-latency buffers. However, in massively parallel SIMT programs found in real-world applications, when the memory access pattern is unknown at compile time, these buffers must be sized for worst-case demand, potentially requiring DRAM-scale storage that is impractical on-chip. Flow-controlled CGRAs (Plasticine [43], SNAFU [13], Riptide [14]) use handshaking protocols between PEs to tolerate variable latency. However, PEs stall when waiting for operands, potentially blocking for hundreds of cycles on cache misses. Dynamic dataflow architectures (SGMF [52], VGIW [53], dMT-CGRA [54]) employ a tagged-token mechanism to achieve non-blocking, out-of-order processing, but at the cost of substantial hardware overhead. In

contrast, DICE’s *p-graph* partitioning sidesteps these issues by separating memory consumers from producers to different configurations at compile time, preserving a cache-based memory system without the need for elastic interconnect between PEs.

CGRA for SIMT: CGRA has been previously applied to the SIMT model. SGMF [52] uses a dynamically scheduled dataflow CGRA with tagged tokens to pipeline multiple threads through a shared dataflow graph. VGIW [53] extends SGMF by introducing control flow coalescing and also a von Neumann-style control to orchestrate basic block execution. dMT-CGRA [54] further adds fine-grained inter-thread communication directly through the CGRA fabric. As described before, they all use dynamic tagged-token CGRAs instead of minimal-overhead statically scheduled CGRAs. DICE applies *p-graph* partitioning and further introduces thread unrolling and the TMCU, which are not explored in these prior works.

Decoupled CGRA Execution and Memory Operation: Softbrain [32] separates memory operations from computation in streaming dataflow architectures but focuses on stream access patterns rather than general-purpose SIMT execution. DICE adopts a different approach with *p-graphs* where the CGRA generates memory requests and forwards them directly to the load/store unit, supporting arbitrary access patterns.

More CGRAs/Processor Integration: ADRES [28] integrates a CGRA with a VLIW processor, sharing functional units and register files between the two modes. DySER [15] tightly couples a CGRA into a CPU pipeline. TRIPS [45] employs a tiled architecture that composes simple cores into larger logical units, targeting both instruction-level and thread-level parallelism. WaveScalar [48] is a dataflow architecture that distributes execution across a grid of processing elements. All these systems explore different ways to integrate CGRAs into CPU-like environments, focusing on composability, dynamic specialization, or single-thread performance. In contrast, DICE is tailored to harness massive thread-level parallelism and memory patterns typical in GPGPU workloads.

VIII. SUMMARY

We presented DICE, an architecture that adopts statically scheduled CGRAs to address critical energy inefficiencies in conventional GPGPUs. Our evaluation shows DICE CPs achieve a geometric mean of $1.77\text{--}1.90\times$ dynamic energy efficiency, 42.0%–45.9% dynamic power reduction relative to NVIDIA Turing SMs, and performance comparable to the modeled NVIDIA Turing GPU baselines under matched compute-resource counts and total cache/SRAM capacity, providing a practical path forward for efficient SIMT computing.

ACKNOWLEDGMENT

We thank the anonymous reviewers and the shepherd for their insightful comments and suggestions. We also thank the DICE RTL team members—Aadithya Manoj, Albert Ton, Elliot Norman, Juwon Jun, Patrick Howe, Rambod Taherian, and Siboz Zhang—for their contributions to the development of the DICE RTL. We also used ChatGPT for light editorial assistance to improve sentence clarity and readability.

A. Abstract

This artifact contains the code, benchmark inputs, DICE metadata bundles, and plotting scripts used to evaluate DICE against baseline GPGPU-Sim. It supports three experiment groups from the paper: base comparison, scale-up, and scale-out. The intended artifact-evaluation path is a full rerun of the experiments followed by plot generation.

B. Artifact Checklist (Meta-Information)

- **Program:** Bash scripts, Python scripts, CUDA Rodinia benchmarks, GPGPU-Sim
- **Compilation:** `make`, `gcc/g++`, `nvcc`
- **Data set:** Bundled Rodinia inputs
- **Run-time environment:** 64-bit Linux
- **Hardware:** Multicore CPU; no physical GPU required
- **Metrics:** Speedup; normalized RF accesses
- **Output:** CSV, PDF, PNG
- **Experiments:** Base; scale-up; scale-out
- **How much disk space required (approximately)?** About 15–20 GB
- **How much time is needed to prepare workflow (approximately)?** About 15–30 minutes
- **How much time is needed to complete experiments (approximately)?** Several hours for a full rerun
- **Publicly available?** Yes
- **Workflow automation framework used:** Bash, `make`, and Python
- **Archived (provide DOI):** <https://doi.org/10.5281/zenodo.19278715>

C. Description

1) *How to Access:* The archived artifact is available at <https://doi.org/10.5281/zenodo.19278715>. A source mirror is available at <https://github.com/jiayi-wang98/DICE-test-collection>. If the GitHub repository is used instead of the archive, initialize the submodules before running anything:

```
git clone https://github.com/\
jiayi-wang98/\
DICE-test-collection.git
cd DICE-test-collection
git submodule update --init --recursive
```

2) *Hardware Dependencies:* A standard 64-bit Linux machine is sufficient. A physical NVIDIA GPU is not required because the experiments are simulation-based. A multicore CPU is recommended because the scripts use parallel builds by default.

3) *Software Dependencies:* The artifact requires bash, GNU make, `gcc/g++`, and the NVIDIA CUDA Toolkit 11.7 (including `nvcc` and `ptxas`). It also requires Python 3 with `numpy` and `matplotlib`. The practical Linux package dependencies are the usual GPGPU-Sim requirements: `build-essential`, `xutils-dev`, `bison`, `flex`, `zlib1g-dev`, `libglu1-mesa-dev`, `libxi-dev`, `libxmu-dev`, and `libglut3-dev`. In practice, a compatible `libasan` runtime is also required because several

benchmarks are built with AddressSanitizer. No proprietary EDA tools are required.

4) *Data Sets:* All benchmark inputs needed by the automated workflow are already bundled in the artifact under `dice-test-gpu-rodinia/data` and `gpu-rodinia/data`. The DICE runs also require the bundled metadata/PPTX directories `dice-test-gpu-rodinia/cuda/dice_test/sw_*`.

5) *Models:* The artifact does not contain a machine-learning model. The relevant configuration models are the included simulator configuration files for the evaluated RTX2060S-, RTX3070-, RTX5000-, and RTX6000-style systems.

D. Installation

Run all commands from the repository root. Set the CUDA paths before running the workflow:

```
export CUDA_INSTALL_PATH=\
/usr/local/cuda-11.7
export PTXAS_CUDA_INSTALL_PATH=\
/usr/local/cuda-11.7
```

On Ubuntu-like systems, the required host packages can be installed with:

```
sudo apt-get install git xutils-dev bison \
flex zlib1g-dev libglu1-mesa-dev \
libxi-dev libxmu-dev libasan5 python3 \
python3-numpy python3-matplotlib
```

On RHEL/Rocky/Alma-like systems, the equivalent packages can be installed with:

```
sudo dnf install git gcc gcc-c++ make \
bison flex zlib-devel \
mesa-libGLU-devel libXi-devel \
libXmu-devel freeglut-devel \
python3 python3-numpy python3-matplotlib
```

If the AddressSanitizer runtime is missing on those systems, install `libasan`.

If the local CUDA installation path differs from `/usr/local/cuda-11.7`, update both `dice-test-gpu-rodinia/common/make.config` and `gpu-rodinia/common/make.config` accordingly.

E. Experiment Workflow

The intended artifact-evaluation path is a full rerun:

A prebuilt Docker image is also available at `jiayiwang0710/dice-isca-eval:cuda11.7`. Reviewers can pull the image and enter an interactive shell with:

```
docker pull
→ jiayiwang0710/dice-isca-eval:cuda11.7
docker run --rm -it
→ jiayiwang0710/dice-isca-eval:cuda11.7
→ shell
```

Inside the container, run:

```
bash integrated_test/run_base_test.sh
bash integrated_test/run_scale_out_test.sh
```

The first script builds and runs the base and scale-up experiments. The second script builds and runs the scale-out experiments. Generated outputs are written to `integrated_test/generated_base`, `integrated_test/generated_scale_up`, and `integrated_test/generated_scale_out`.

F. Evaluation and Expected Results

The main generated PDFs corresponding to the paper figures are:

- Figure 9: `integrated_test/generated_base/rf_access_reduction.pdf`
- Figure 10: `integrated_test/generated_base/dice_gpu_speedup.pdf`
- Figure 15: `integrated_test/generated_scale_up/40PE_vs_20PE_perf.pdf` and `integrated_test/generated_scale_up/40PE_vs_20PE_rf.pdf`
- Figure 16: `integrated_test/generated_scale_out/scale_out_speedup.pdf`
- Figure 18: `integrated_test/generated_scale_out/scale_out_3070_speedup.pdf` and `integrated_test/generated_scale_out/scale_out_3070_rf_reduction.pdf`

The corresponding summary CSV files are stored in the same output directories.

G. Experiment Customization

Reviewers can override `JOBS=<n>` to control parallelism, and can selectively enable or disable parts of the workflow using the Boolean environment variables already exposed by the two top-level wrapper scripts.

H. Notes

This appendix documents the fresh-machine full-rerun path. It intentionally does not rely on regenerating plots from pre-existing results.

REFERENCES

- [1] T. M. Aamodt, W. W. L. Fung, and T. G. Rogers, *General-Purpose Graphics Processor Architectures*, 1st ed., ser. Synthesis Lectures on Computer Architecture. Cham: Springer, 2018.
- [2] O. Antepara, Z. Zhao, B. Austin, N. Ding, L. Oliker, N. J. Wright, and S. Williams, "Benchmark-driven models for energy analysis and attribution of GPU-accelerated supercomputing," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 888–904. [Online]. Available: <https://doi.org/10.1145/3712285.3759815>
- [3] Arvind and R. Nikhil, "Executing a program on the MIT tagged-token dataflow architecture," *IEEE Transactions on Computers*, vol. 39, no. 3, pp. 300–318, 1990.
- [4] T. M. Bruintjes, "Design of a fused multiply-add floating-point and integer datapath," Master's thesis, University of Twente, Enschede, The Netherlands, 2011. [Online]. Available: https://essay.utwente.nl/fileshare/file/61055/MSc_TM_Bruintjes_CAES_ASCII.pdf#page=121.00
- [5] A. Burnes, "GeForce GTX 1660 Ti's Advanced Shaders Accelerate Performance In The Latest Games," *NVIDIA GeForce News*, Feb. 2019, accessed March 28, 2026. [Online]. Available: <https://www.nvidia.com/en-us/geforce/news/geforce-gtx-1660-ti-advanced-shaders-streaming-multiprocessor/>
- [6] Cadence Design Systems, Inc., 2025. [Online]. Available: https://www.cadence.com/en_US/home/tools/digital-design-and-signoff/power-analysis/joules-rtl-power-solution.html
- [7] S. Che, M. Boyer, J. Meng, D. Tarjan, J. W. Sheaffer, S.-H. Lee, and K. Skadron, "Rodinia: A benchmark suite for heterogeneous computing," in *2009 IEEE International Symposium on Workload Characterization (IISWC)*, 2009, pp. 44–54.
- [8] S. A. Chin, N. Sakamoto, A. Rui, J. Zhao, J. H. Kim, Y. Hara-Azumi, and J. Anderson, "CGRA-ME: A unified framework for CGRA modelling and exploration," in *2017 IEEE 28th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*, 2017, pp. 184–189.
- [9] K. Feng, T. Kong, K. Koul, J. Melchert, A. Carsello, Q. Liu, G. Nyengele, M. Strange, K. Zhang, A. Nayak, J. Setter, J. Thomas, K. Sreedhar, P.-H. Chen, N. Bhagdikar, Z. A. Myers, B. D'Agostino, P. Joshi, S. Richardson, C. Tornig, M. Horowitz, and P. Raina, "Amber: A 16-nm system-on-chip with a coarse-grained reconfigurable array for flexible acceleration of dense linear algebra," *IEEE Journal of Solid-State Circuits*, vol. 59, no. 3, pp. 947–959, 2024.
- [10] F. Fritz, "Nvidia 12nm Turing TU106 GeForce RTX 2060 Die Shot," Flickr, 2019, cC0 1.0 / public domain. [Online]. Available: <https://www.flickr.com/photos/130561288@N04/46357922535>
- [11] —, "Nvidia 12nm Turing TU116 GeForce GTX 1660 Ti Die Shot," Flickr, 2019, cC0 1.0 / public domain. [Online]. Available: <https://www.flickr.com/photos/130561288@N04/47220396142>
- [12] W. W. Fung, I. Sham, G. Yuan, and T. M. Aamodt, "Dynamic warp formation and scheduling for efficient GPU control flow," in *40th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO 2007)*, 2007, pp. 407–420.
- [13] G. Gobieski, A. O. Atli, K. Mai, B. Lucia, and N. Beckmann, "SNAFU: An ultra-low-power, energy-minimal CGRA-generation framework and architecture," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, 2021, pp. 1027–1040.
- [14] G. Gobieski, S. Ghosh, M. Heule, T. Mowry, T. Nowatzki, N. Beckmann, and B. Lucia, "RipTide: A programmable, energy-minimal dataflow compiler and architecture," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, pp. 546–564.
- [15] V. Govindaraju, C.-H. Ho, T. Nowatzki, J. Chhugani, N. Satish, K. Sankaralingam, and C. Kim, "DySER: Unifying functionality and parallelism specialization for energy-efficient computing," *IEEE Micro*, vol. 32, no. 5, pp. 38–51, 2012.
- [16] R. Hameed, W. Qadeer, M. Wachs, O. Azizi, A. Solomatnikov, B. C. Lee, S. Richardson, C. Kozyrakis, and M. Horowitz, "Understanding sources of inefficiency in general-purpose chips," p. 37–47, 2010. [Online]. Available: <https://doi.org/10.1145/1815961.1815968>
- [17] S. Hauck and A. DeHon, Eds., *Reconfigurable Computing: The Theory and Practice of FPGA-Based Computation*, ser. Systems on Silicon Ser. San Francisco: Morgan Kaufmann, 2007.
- [18] S. Hong and H. Kim, "An integrated GPU power and performance model," in *Proceedings of the 37th Annual International Symposium on Computer Architecture*, ser. ISCA '10. New York, NY, USA: Association for Computing Machinery, 2010, p. 280–289. [Online]. Available: <https://doi.org/10.1145/1815961.1815998>
- [19] V. Kandiah, S. Peverelle, M. Khairy, J. Pan, A. Manjunath, T. G. Rogers, T. M. Aamodt, and N. Hardavellas, "AccelWatch: A power modeling framework for modern GPUs," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 738–753. [Online]. Available: <https://doi.org/10.1145/3466752.3480063>
- [20] M. Karunaratne, A. K. Mohite, T. Mitra, and L.-S. Peh, "HyCUBE: A CGRA with reconfigurable single-cycle multi-hop interconnect," in *2017 54th ACM/EDAC/IEEE Design Automation Conference (DAC)*, 2017, pp. 1–6.
- [21] M. Khairy, Z. Shen, T. M. Aamodt, and T. G. Rogers, "Accel-Sim: An extensible simulation framework for validated GPU modeling," in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020, pp. 473–486.
- [22] M. Khazraee, L. Zhang, L. Vega, and M. B. Taylor, "Moonwalk: NRE optimization in ASIC clouds," in *Proceedings of the Twenty-Second*

- International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '17. New York, NY, USA: Association for Computing Machinery, 2017, p. 511–526. [Online]. Available: <https://doi.org/10.1145/3037697.3037749>
- [23] K. Koul, J. Melchert, K. Sreedhar, L. Truong, G. Nyengele, K. Zhang, Q. Liu, J. Setter, P.-H. Chen, Y. Mei, M. Strange, R. Daly, C. Donovanick, A. Carsello, T. Kong, K. Feng, D. Huff, A. Nayak, R. Setaluri, J. Thomas, N. Bhagdikar, D. Durst, Z. Myers, N. Tsiskaridze, S. Richardson, R. Bahr, K. Fatahalian, P. Hanrahan, C. Barrett, M. Horowitz, C. Torng, F. Kjolstad, and P. Raina, “AHA: An agile approach to the design of coarse-grained reconfigurable accelerators and compilers,” *ACM Trans. Embed. Comput. Syst.*, vol. 22, no. 2, Jan. 2023. [Online]. Available: <https://doi.org/10.1145/3534933>
- [24] D. Kroft, “Lockup-free instruction fetch/prefetch cache organization,” in *Proceedings of the 8th Annual Symposium on Computer Architecture*, ser. ISCA '81. Washington, DC, USA: IEEE Computer Society Press, 1981, p. 81–87.
- [25] J. Leng, T. Hetherington, A. ElTantawy, S. Gilani, N. S. Kim, T. M. Aamodt, and V. J. Reddi, “GPUWattch: enabling energy optimizations in GPGPUs,” in *Proceedings of the 40th Annual International Symposium on Computer Architecture*, ser. ISCA '13. New York, NY, USA: Association for Computing Machinery, 2013, p. 487–498. [Online]. Available: <https://doi.org/10.1145/2485922.2485964>
- [26] E. Lindholm, J. Nickolls, S. Oberman, and J. Montrym, “NVIDIA Tesla: A unified graphics and computing architecture,” *IEEE Micro*, vol. 28, no. 2, pp. 39–55, 2008.
- [27] L. Liu, J. Zhu, Z. Li, Y. Lu, Y. Deng, J. Han, S. Yin, and S. Wei, “A survey of coarse-grained reconfigurable architecture and design: Taxonomy, challenges, and applications,” *ACM Comput. Surv.*, vol. 52, no. 6, Oct. 2019. [Online]. Available: <https://doi.org/10.1145/3357375>
- [28] B. Mei, S. Vernalde, D. Verkest, H. De Man, and R. Lauwereins, “ADRES: An architecture with tightly coupled VLIW processor and coarse-grained reconfigurable matrix,” in *Field Programmable Logic and Application*, P. Y. K. Cheung and G. A. Constantinides, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2003, pp. 61–70.
- [29] J. Melchert, K. Feng, C. Donovanick, R. Daly, C. Barrett, M. Horowitz, P. Hanrahan, and P. Raina, “Automated design space exploration of CGRA processing element architectures using frequent subgraph analysis,” 2021. [Online]. Available: <https://arxiv.org/abs/2104.14155>
- [30] N. Muralimanohar, R. Balasubramanian, and N. Jouppi, “Optimizing NUCA organizations and wiring alternatives for large caches with CACTI 6.0,” in *40th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO 2007)*, 2007, pp. 3–14.
- [31] J. Nickolls, I. Buck, M. Garland, and K. Skadron, “Scalable parallel programming with CUDA: Is CUDA the parallel programming model that application developers have been waiting for?” *Queue*, vol. 6, no. 2, pp. 40–53, Mar. 2008. [Online]. Available: <https://doi.org/10.1145/1365490.1365500>
- [32] T. Nowatzki, V. Gangadhar, N. Ardalani, and K. Sankaralingam, “Stream-dataflow acceleration,” in *2017 ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA)*, 2017, pp. 416–429.
- [33] NVIDIA Corporation, “NVIDIA Tesla P100,” <https://images.nvidia.com/content/pdf/tesla/whitepaper/pascal-architecture-whitepaper.pdf>, 2016, white paper.
- [34] —, “NVIDIA TESLA V100 GPU ARCHITECTURE,” <https://images.nvidia.com/content/volta-architecture/pdf/volta-architecture-whitepaper.pdf>, 2017, white paper.
- [35] —, “NVIDIA TURING GPU ARCHITECTURE,” Tech. Rep., NVIDIA, Santa Clara, CA, Tech. Rep., 2018, <https://images.nvidia.com/aem-dam/en-zz/Solutions/design-visualization/technologies/turing-architecture/NVIDIA-Turing-Architecture-Whitepaper.pdf>.
- [36] —, “NVIDIA A100 Tensor Core GPU Architecture,” <https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/nvidia-ampere-architecture-whitepaper.pdf>, 2020, white paper.
- [37] —, “NVIDIA Ampere GA102 GPU Architecture,” <https://www.nvidia.com/content/PDF/nvidia-ampere-ga-102-gpu-architecture-whitepaper-v2.pdf>, 2020, white paper.
- [38] —, “NVIDIA H100 Tensor Core GPU Architecture,” <https://resources.nvidia.com/en-us-hopper-architecture/nvidia-h100-tensor-c>, 2022, white paper.
- [39] —, *Parallel Thread Execution (PTX) ISA Version 8.8*, 2024, <https://docs.nvidia.com/cuda/parallel-thread-execution/>.
- [40] —, “CUDA Toolkit Documentation: NVCC, the CUDA Compiler Driver,” <https://docs.nvidia.com/cuda/cuda-compiler-driver-nvcc/index.html>, June 2025, accessed: 2025-07-30.
- [41] P. M. Phothilimthana, A. S. Elliott, A. Wang, A. Jangda, B. Hagedorn, H. Barthels, S. J. Kaufman, V. Grover, E. Torlak, and R. Bodik, “Swizzle Inventor: Data movement synthesis for GPU kernels,” in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 65–78. [Online]. Available: <https://doi.org/10.1145/3297858.3304059>
- [42] A. Podobas, K. Sano, and S. Matsuoka, “A survey on coarse-grained reconfigurable architectures from a performance perspective,” *IEEE Access*, vol. 8, pp. 146 719–146 743, 2020.
- [43] R. Prabhakar, Y. Zhang, D. Koeplinger, M. Feldman, T. Zhao, S. Hadjis, A. Pedram, C. Kozyrakis, and K. Olukotun, “Plasticine: A reconfigurable architecture for parallel patterns,” in *2017 ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA)*, 2017, pp. 389–402.
- [44] R. Raj, S. Banerjee, N. Chandra, Z. Wan, J. Tong, A. Samajdhara, and T. Krishna, “SCALE-Sim V3: a modular cycle-accurate systolic accelerator simulator for end-to-end system analysis,” in *2025 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2025, pp. 186–200.
- [45] K. Sankaralingam, R. Nagarajan, H. Liu, C. Kim, J. Huh, D. Burger, S. Keckler, and C. Moore, “Exploiting ILP, TLP, and DLP with the polymorphous TRIPS architecture,” in *30th Annual International Symposium on Computer Architecture*, 2003. *Proceedings.*, 2003, pp. 422–433.
- [46] A. Stillmaker and B. Baas, “Scaling equations for the accurate prediction of CMOS device performance from 180 nm to 7 nm,” *Integration*, vol. 58, pp. 74–81, 2017. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167926017300755>
- [47] J. E. Stine, I. Castellanos, M. Wood, J. Henson, F. Love, W. R. Davis, P. D. Franzon, M. Bucher, S. Basavarajaiah, J. Oh, and R. Jenkal, “FreePDK: An open-source variation-aware design kit,” in *2007 IEEE International Conference on Microelectronic Systems Education (MSE'07)*, 2007, pp. 173–174.
- [48] S. Swanson, K. Michelson, A. Schwerin, and M. Oskin, “WaveScalar,” in *Proceedings. 36th Annual IEEE/ACM International Symposium on Microarchitecture*, 2003. *MICRO-36*, 2003, pp. 291–302.
- [49] C. Tan, D. Patil, A. Tumeo, G. Weisz, S. Reinhardt, and J. Zhang, “VecPAC: A vectorizable and precision-aware CGRA,” in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, 2023, pp. 1–9.
- [50] C. Tan, C. Xie, A. Li, K. J. Barker, and A. Tumeo, “OpenCGRA: An open-source unified framework for modeling, testing, and evaluating CGRAs,” in *2020 IEEE 38th International Conference on Computer Design (ICCD)*, 2020, pp. 381–388.
- [51] T. N. Theis and H.-S. P. Wong, “The end of Moore’s law: A new beginning for information technology,” *Computing in Science & Engineering*, vol. 19, no. 2, pp. 41–50, 2017.
- [52] D. Voitsechov and Y. Etsion, “Single-graph multiple flows: energy efficient design alternative for GPGPUs,” p. 205–216, 2014.
- [53] —, “Control flow coalescing on a hybrid dataflow/von Neumann GPGPU,” in *2015 48th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2015, pp. 216–227.
- [54] D. Voitsechov, O. Port, and Y. Etsion, “Inter-Thread Communication in Multithreaded, Reconfigurable Coarse-Grain Arrays,” in *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2018, pp. 42–54.
- [55] I. Watson and J. Gurd, “A prototype data flow computer with token labelling,” in *1979 International Workshop on Managing Requirements Knowledge (MARK)*, 1979, pp. 623–628.
- [56] J. Weng, S. Liu, Z. Wang, V. Dadu, and T. Nowatzki, “A hybrid systolic-dataflow architecture for inductive matrix algorithms,” in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2020, pp. 703–716.